

Abdessamad Ait Elmouden, Jihane Erraji, Islam Bouikiri

CSC 3309 01

Dr. Driss Kettani

Mini project 2: Checkers-Agent

1. Problem & Design Overview

This project implements a fully playable Checkers game in Python, combining a Tkinter graphical user interface with three distinct AI search algorithms. The human player (White) faces off against an AI opponent (Black) that can use Minimax, Alpha-Beta Pruning, or Alpha-Beta with Move Ordering, selectable at game start along with depth and time-limit settings.

The architecture is split across four Python modules for clarity and maintainability:

- `PlayingTheGame.py` — entry point; initializes the game and coordinates turns
- `OtherStuff.py` — constants, `GameMove` class, `AnalyticsTracker`, `AnalyticsManager`, `GUI`
- `GameBoard.py` — board state, move validation, captures, king promotion, win detection
- `SearchToolBox.py` — all three AI algorithms and the heuristic evaluation function

The three primary design goals were:

- Correctness: valid checkers moves, captures, multi-jumps, and win/stalemate detection
- Strategic AI: depth-limited search with pruning and a weighted heuristic for move quality
- Clean GUI: colored board, clickable move highlighting, and a real-time AI thought-process log

2. Environment and Rules

The game is played on a standard 8x8 Checkers board. Each cell holds one of five states: White piece (W), Black piece (B), White King (w), Black King (b), or Empty (.). The game follows standard checkers rules:

- Pieces move diagonally forward only; kings move in all four diagonal directions
- Jumps (captures) are available to the player whenever an opponent piece can be leaped
- Reaching the opponent's back row promotes a piece to king
- The game ends when one player has no pieces remaining or has no legal moves (stalemate)

The AI always plays as Black. The human plays as White and moves first. Turns alternate until a terminal state is reached. The GUI highlights valid target squares for each selected piece in real time.

3. AI Algorithms

3.1 Minimax

The Minimax algorithm performs a full depth-first tree search to a configurable maximum depth. It alternates between maximizing (AI / Black) and minimizing (Human / White) layers. At each leaf node (depth 0 or terminal state), the heuristic evaluation function assigns a score. The algorithm propagates scores back up the tree, always selecting the move that leads to the highest guaranteed outcome for the AI.

Because Minimax explores every branch, it is the slowest of the three strategies but also the most exhaustive within its depth. With depth 9 and a 3-second time limit, it expanded an average of **~14,100 nodes per move in testing**.

3.2 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimized version of Minimax that maintains two bounds during the search: alpha (the best value the maximizer can guarantee) and beta (the best value the minimizer can guarantee). Whenever $\beta \leq \alpha$, the current branch is cut off because the opponent would never allow the game to reach this state.

This dramatically reduces the number of nodes evaluated in best-case scenarios it can halve the effective branching factor. In the game log for the Alpha-Beta run, **a total of 78,854 nodes were pruned across 33 AI moves (404,611 nodes expanded), achieving meaningful savings over Minimax's 310,941 nodes across 22 moves**.

3.3 Alpha-Beta with Move Ordering

Move ordering improves Alpha-Beta by sorting candidate moves before exploring them, so the most promising moves (captures, promotions, center-control moves) are evaluated first. This increases the likelihood that alpha-beta cutoffs occur earlier, maximizing pruning effectiveness.

The ordering heuristic prioritizes: **(1) jumps/captures (+100 priority), (2) king-promoting moves (+50), (3) center proximity, and (4) advancement toward promotion**. In the test game using this strategy, **83,750 nodes were pruned out of a combined 423,785 expanded across 28 AI moves** achieving slightly better pruning efficiency than plain Alpha-Beta on this game sequence

4. Heuristic Evaluation Function

The heuristic function (EvaluateBoardState in SearchToolBox.py) estimates the favorability of a board state from the perspective of a given player. It is computed at every leaf node of the search tree. The overall score is:

$$H = \text{player_score} - \text{opponent_score} + \text{mobility_score}$$

Each piece on the board is assigned an individual value that accumulates into the player or opponent score. The per-piece value is built up from three components:

4.1 Base Material Value

Every piece starts with a base value depending on its type:

- Regular piece: 3.0 points
- King: 5.0 points (king bonus replaces the base value entirely)

4.2 Positional / Advancement Bonus (regular pieces only)

Non-king pieces receive a bonus for being closer to the opponent's back row (promotion row). The closer a piece is to promotion, the more strategically valuable it is:

$$\begin{aligned} \text{advancement} &= \text{row} / 7 \text{ (BLACK) or } (7 - \text{row}) / 7 \text{ (WHITE)} \\ \text{advancement_bonus} &= \text{advancement} \times 0.5 \end{aligned}$$

This gives a maximum bonus of +0.5 for a piece one step away from becoming a king, and 0 for a piece on its own back row.

4.3 Center Control Bonus

Pieces that occupy squares closer to the center of the board receive a bonus. The center is defined as position (3.5, 3.5), and the bonus decreases linearly with Manhattan distance from that center:

$$\begin{aligned} \text{center_distance} &= |\text{row} - 3.5| + |\text{col} - 3.5| \\ \text{center_bonus} &= (7.0 - \text{center_distance}) \times 0.1 \end{aligned}$$

A piece in the exact center scores the maximum center bonus (~0.7), while a corner piece scores the minimum (~0.0). This encourages the AI to control the board's interior.

4.4 Mobility Score

After all piece scores are summed up, a separate mobility term is added to the total. Mobility is defined as the number of legal moves available. The mobility score rewards having more options than the opponent:

$$\text{mobility_score} = (\text{player_legal_moves} - \text{opponent_legal_moves}) \times 0.2$$

4.5 Summary Table

Component	Applied To	Formula / Value	Max Contribution
Base material	All pieces	3.0 (regular) or 5.0 (king)	+5.0 per piece
Advancement bonus	Regular pieces only	advancement x 0.5	+0.5 per piece

Center control	All pieces	$(7.0 - \text{manhattan_dist}) \times 0.1$	$\sim +0.7$ per piece
Mobility	Whole board	$(\text{player_moves} - \text{opp_moves}) \times 0.2$	Variable

A positive H value means the board favors the current player; a negative value favors the opponent. The weights (0.5, 0.1, 0.2) were chosen so that material advantage dominates, advancement provides a secondary incentive to advance pieces, and center control and mobility provide fine-grained positional guidance.

5. Code–Spec Alignment

File	Responsibility
PlayingTheGame.py	Entry point. Initializes Tkinter root window, sets up game loop, coordinates human and AI turns, records move history and analytics.
OtherStuff.py	Board constants (BOARD_SIZE, WHITE, BLACK, KING variants, EMPTY), GameMove data class, AnalyticsTracker (per-move stats), AnalyticsManager (cumulative stats).
GameBoard.py	Board state as 2D list. is_valid_move(), make_move(), has_jump(), has_any_legal_move(), setup_initial_board(), king promotion on back-row arrival.
SearchToolBox.py	MinimaxSearch, AlphaBetaSearch, AlphaBetaWithOrdering classes each with GetBestMove() and recursive helpers. EvaluateBoardState() heuristic. GetAllLegalMoves() and ApplyMove() utilities.
checkersGui.py	Tkinter GUI. Setup dialog (mode, algorithm, depth). Board drawing with colored squares, piece circles, king crowns. Click handler for selection and moves. AI thought-process log panel.

6. GUI and Interaction

The graphical interface is built with Tkinter. On launch, a setup dialog appears first (before the board) where the user configures:

- Game mode: Human vs Human or Human vs AI
- AI algorithm: Minimax, Alpha-Beta, or Alpha-Beta + Move Ordering
- Search depth: slider from 3 to 9 plies

After clicking Start, the main board window appears. The board uses alternating light/dark brown squares. White pieces appear as light circles with a gold rim; Black pieces as dark circles with a silver rim. Kings

display a crown symbol (♔). A selected piece is highlighted in green; valid destinations show colored dots, red for captures, blue for regular moves.

The right panel displays a live AI Thought Process log. After every AI move, the log appends: algorithm name, move coordinates, nodes expanded, nodes pruned, maximum depth reached, and elapsed time. This allows direct comparison of algorithm behavior mid-game.

7. Experimental Results

Three full games were played (Human vs AI) at depth 9 with a 3-second time limit per move, one for each algorithm. All three games ended with Black (AI) winning.

7.1 Game Summaries

Algorithm	Total Moves	AI Moves	Total Nodes Expanded	Total Nodes Pruned	Total Search Time	Avg Time / Move
Minimax	44	22	310,941	0	63.014s	2.864s
Alpha-Beta	68	33	404,611	78,854	71.208s	2.158s
Alpha-Beta + Ordering	56	28	423,785	83,750	73.370s	2.620s

7.2 Per-Move Node Expansion (Selected Moves)

The table below shows nodes expanded for the first six AI moves in each game, illustrating how early-game complexity evolves across algorithms:

Move #	Minimax (nodes)	Alpha-Beta (nodes)	AB+Ordering (nodes)
2	13,114	13,366	13,764
4	13,297	13,204	14,147
6	13,792	13,993	14,619
8	13,280	14,458	14,483
10	13,697	14,882	14,705
12	14,603	15,186	14,983

7.3 Observations

- Minimax expanded zero pruned nodes (as expected) and used the most time per move at depths where it exhausted the time limit.
- Alpha-Beta pruned 78,854 nodes (approximately 19.5% of the combined node count), reducing per-move search time meaningfully in the mid-game.
- Alpha-Beta with Move Ordering pruned 83,750 nodes (19.8%), slightly outperforming plain Alpha-Beta — the ordering heuristic causes cutoffs to trigger earlier.
- All three algorithms consistently reached depth 9, confirming the time-limit fallback works correctly.
- On moves where the outcome was nearly forced (e.g., the final move in each game with only one legal move), all algorithms returned in under 0.002 seconds, demonstrating efficient early termination.

7.4 Time and Space Complexity

Time complexity. All three strategies are bounded by the same worst-case time complexity of $O(B^P)$, where B is the branching factor and P is the search depth. In practice:

- Minimax always reaches this worst case, expanding every node to depth 9
- Alpha-Beta achieves approximately $O(gPR)$ in the best case (perfectly ordered moves), reducing the effective branching factor. The observed -19.5% node reduction in this game is consistent with a moderately ordered search tree.
- AB + Ordering approaches the best case more closely, achieving -19.8% pruning by prioritizing captures and promotions first.
- Space complexity. All three strategies use iterative depth-first traversal, so the call stack at any point holds at most $P = 9$ levels of board states. Space complexity is therefore $O(P \times B)$, where B is the branching factor at a given level. Pruning does not reduce space complexity - only time complexity - because the stack is still traversed depth-first. The use of `copy.deepcopy()` in `ApplyMove()` means each node in the stack holds a full 8x8 board copy, making space usage $O(64 \times P) = O(576)$ board cells at peak, which is negligible.

8. Debugging and Testing

Several issues were identified and resolved during development:

- Tkinter 8-digit hex color error: Python 3.13's Tkinter does not support #RRGGBBAA alpha colors. Shadow effects were replaced with stipple patterns (gray25) to simulate transparency.
- Board appearing before setup dialog: the main Tk window was hidden with `root.withdraw()` on launch and only revealed after the user clicked Start in the setup dialog.
- Mandatory jump filter blocking all moves: the GUI was incorrectly hiding non-jump moves when any jump existed. The filter was removed to show all legal moves.

9. How to Run

Run the game using: `python main.py` Select search algorithm (Minimax, Alpha-Beta, or Alpha-Beta Ordering), time limit, and depth, then click 'Start Game'. White (human) moves first using mouse clicks; the AI responds automatically. You can play against a human if you want to

10. Conclusion

This project successfully implemented a complete, playable Checkers game with three AI search strategies. The results confirm that Alpha-Beta pruning and Alpha-Beta ordering provide measurable efficiency gains over Minimax, reducing effective search space by approximately 19% in these test games while maintaining or improving move quality.

The modular code structure (separate files for board logic, AI search, and GUI) proved beneficial during debugging and extension. The live AI thought-process log gives a transparent view of how each algorithm reasons, making the project valuable as both a playable game and an educational demonstration of classical adversarial search.

Future improvements could include iterative deepening (to better exploit the time limit), transposition tables (to avoid re-evaluating repeated board states), and an endgame database for perfect play in late-game scenarios.

This project gave us hands-on experience with search algorithms, game heuristics, and GUI programming. We learned how decision trees power intelligent gameplay and how pruning drastically reduces search time. Despite minor GUI polish left, the project was a success, combining logical precision with interactive design. It was a very fruitful experience.